

backend/server.js

```
const express = require('express');
const cors = require('cors');

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware
app.use(cors());
app.use(express.json());

// Sample data
const projects = [
  {
    id: 1,
    title: 'CI/CD Portfolio',
    description: 'A fully automated portfolio deployed on Kubernetes using Jenkins and Docker.',
    technologies: ['React', 'Jenkins', 'Docker', 'Kubernetes']
  },
  {
    id: 2,
    title: 'Sample Microservice',
    description: 'Containerized Node.js API with Kubernetes deployment and monitoring.',
    technologies: ['Node.js', 'Express', 'Docker', 'Kubernetes', 'Prometheus']
  },
  {
    id: 3,
    title: 'Monitoring Dashboard',
    description: 'Basic monitoring setup using Prometheus & Grafana for container workloads.',
    technologies: ['Prometheus', 'Grafana', 'Docker', 'Kubernetes']
  }
];

const skills = [
  {
    category: 'DevOps & Cloud',
    items: ['Jenkins', 'Docker', 'Kubernetes', 'Helm', 'GitHub Actions']
  },
  {
    category: 'Backend',
    items: ['Node.js', 'Express', 'REST APIs']
  },
  {
    category: 'Frontend',
    items: ['React', 'HTML', 'CSS', 'JavaScript']
  },
  {
    category: 'Tools',
    items: ['Linux', 'Git', 'Bash', 'Docker Hub', 'Minikube']
  }
];

// Routes
app.get('/api/health', (req, res) => {
  res.json({ status: 'OK', message: 'Backend API is running' });
});

app.get('/api/projects', (req, res) => {
  res.json(projects);
});
```

backend/server.js (cont.)

```
app.get('/api/skills', (req, res) => {
  res.json(skills);
});

app.get('/api/projects/:id', (req, res) => {
  const project = projects.find(p => p.id === parseInt(req.params.id));
  if (project) {
    res.json(project);
  } else {
    res.status(404).json({ error: 'Project not found' });
  }
});

// Start server
app.listen(PORT, () => {
  console.log('Backend server running on port ${PORT}');
});
```

backend/package.json

```
{
  "name": "portfolio-frontend",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.0",
    "react-dom": "^18.3.0",
    "vite": "^6.4.1",
    "@vitejs/plugin-react": "^4.7.0"
  }
}
```

backend/Dockerfile

```
# Use Node.js runtime
FROM node:20-alpine

# Set working directory
WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm install --only=production

# Copy source code
COPY . .

# Expose port
EXPOSE 3000

# Start the application
CMD ["npm", "start"]
```

frontend/src/App.jsx

```
import React, { useEffect, useState } from "react";
import "../App.css";

function App() {
  const [buildInfo, setBuildInfo] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [projects, setProjects] = useState([]);
  const [projectsLoading, setProjectsLoading] = useState(true);

  useEffect(() => {
    const fetchBuildInfo = async () => {
      try {
        const res = await fetch("/build-info.json", { cache: "no-store" });
        if (!res.ok) {
          throw new Error("Failed to load build info");
        }
        const data = await res.json();
        setBuildInfo(data);
        // #region agent log
        fetch(
          "http://127.0.0.1:7394/ingest/722304ba-72db-4375-9a3c-0f8c249f8c2c",
          {
            method: "POST",
            headers: {
              "Content-Type": "application/json",
              "X-Debug-Session-Id": "f65546",
            },
            body: JSON.stringify({
              sessionId: "f65546",
              runId: "initial",
              hypothesisId: "H1",
              location: "App.jsx:16",
              message: "build-info fetch success",
              data: { buildInfo: data },
              timestamp: Date.now(),
            }),
          },
        ).catch(() => {});
        // #endregion
      } catch (err) {
        // #region agent log
        fetch(
          "http://127.0.0.1:7394/ingest/722304ba-72db-4375-9a3c-0f8c249f8c2c",
          {
            method: "POST",
            headers: {
              "Content-Type": "application/json",
              "X-Debug-Session-Id": "f65546",
            },
            body: JSON.stringify({
              sessionId: "f65546",
              runId: "initial",
              hypothesisId: "H2",
              location: "App.jsx:18",
              message: "build-info fetch failed",
              data: { error: String(err) },
              timestamp: Date.now(),
            }),
          },
        );
      }
    };
  });
}
```

frontend/src/App.jsx (cont.)

```
    ).catch(() => {});
    // #endregion
    setError(err instanceof Error ? err.message : String(err));
  } finally {
    setLoading(false);
  }
};

fetchBuildInfo();
}, []);

useEffect(() => {
  const fetchProjects = async () => {
    try {
      // In development: http://localhost:3000/api/projects
      // In production: http://<backend-service-url>/api/projects
      const res = await fetch('http://localhost:3000/api/projects'); // Change to actual
      backend URL
      if (!res.ok) {
        throw new Error('Failed to load projects');
      }
      const data = await res.json();
      setProjects(data);
    } catch (err) {
      console.error('Failed to fetch projects:', err);
      // Fallback to static data
      setProjects([
        {
          id: 1,
          title: 'CI/CD Portfolio',
          description: 'This project ? a fully automated portfolio deployed on Kubernetes
          using Jenkins and Docker.',
          technologies: ['React', 'Jenkins', 'Docker', 'Kubernetes']
        },
        {
          id: 2,
          title: 'Sample Microservice',
          description: 'Containerized Node.js API with Kubernetes deployment and
          monitoring.',
          technologies: ['Node.js', 'Express', 'Docker', 'Kubernetes', 'Prometheus']
        },
        {
          id: 3,
          title: 'Monitoring Dashboard',
          description: 'Basic monitoring setup using Prometheus & Grafana for container
          workloads.',
          technologies: ['Prometheus', 'Grafana', 'Docker', 'Kubernetes']
        }
      ]);
    } finally {
      setProjectsLoading(false);
    }
  };

  fetchProjects();
}, []);

const formattedDeployTime =
  buildInfo && buildInfo.deployTime
    ? new Date(buildInfo.deployTime).toLocaleString()
```

frontend/src/App.jsx (cont.)

```
: "N/A";

return (
  <div className="app">
    <header className="navbar">
      <div className="logo">DevOps Portfolio</div>
      <nav>
        <a href="#hero">Home</a>
        <a href="#about">About</a>
        <a href="#skills">Skills</a>
        <a href="#projects">Projects</a>
        <a href="#contact">Contact</a>
      </nav>
    </header>

    <main>
      <section id="hero" className="hero">
        <div className="hero-content">
          <p className="tagline">Hello, I'm</p>
          <h1>Vaddeboina Akhil</h1>
          <h2>DevOps Engineer & Full Stack Creator</h2>
          <p className="hero-subtitle">
            Building automated pipelines for modern cloud-native apps with
            Jenkins, Docker, and Kubernetes.
          </p>
          <div className="hero-actions">
            <a href="#projects" className="btn primary">
              Explore My Work
            </a>
            <a href="#contact" className="btn secondary">
              Get in Touch
            </a>
          </div>
        </div>
      </section>

      <section id="about" className="section">
        <h3>About</h3>
        <p>
          This portfolio is fully automated using a CI/CD pipeline. Whenever I
          push code to GitHub, Jenkins builds a new Docker image and deploys
          it to a Kubernetes cluster using Rolling Updates.
        </p>
      </section>

      <section id="skills" className="section">
        <h3>Skills</h3>
        <div className="grid">
          <div className="card">
            <h4>DevOps & Cloud</h4>
            <p>Jenkins, Docker, Kubernetes, Helm, GitHub Actions</p>
          </div>
          <div className="card">
            <h4>Backend</h4>
            <p>Node.js, Express, REST APIs</p>
          </div>
          <div className="card">
            <h4>Frontend</h4>
            <p>React, HTML, CSS, JavaScript</p>
          </div>
        </div>
      </section>
    </main>
  </div>
);
```

frontend/src/App.jsx (cont.)

```
    <div className="card">
      <h4>Tools</h4>
      <p>Linux, Git, Bash, Docker Hub, Minikube</p>
    </div>
  </div>
</section>

<section id="projects" className="section">
  <h3>Projects</h3>
  {projectsLoading ? (
    <p>Loading projects...</p>
  ) : (
    <div className="grid">
      {projects.map(project => (
        <div key={project.id} className="card">
          <h4>{project.title}</h4>
          <p>{project.description}</p>
          {project.technologies && (
            <p><strong>Technologies:</strong> {project.technologies.join(', ')}</p>
          )}
        </div>
      ))}
    </div>
  )}
</section>

<section id="cicd" className="section ci-panel">
  <h3>CI/CD Deployment Info</h3>
  {loading && <p className="status">Loading build information?</p>}
  {error && !loading && (
    <p className="status error">
      Failed to load build info. Showing default values.
    </p>
  )}
  <div className="ci-grid">
    <div className="ci-item">
      <span className="label">App Version</span>
      <span className="value">
        {buildInfo?.appVersion ?? "v1.0.0"}
      </span>
    </div>
    <div className="ci-item">
      <span className="label">Build Number</span>
      <span className="value">
        {buildInfo?.buildNumber ? `#${buildInfo.buildNumber}` : "#0"}
      </span>
    </div>
    <div className="ci-item">
      <span className="label">Last Deployed</span>
      <span className="value">{formattedDeployTime}</span>
    </div>
    <div className="ci-item">
      <span className="label">Environment</span>
      <span className="value">
        {buildInfo?.environment ?? "Kubernetes"}
      </span>
    </div>
  </div>
  <p className="note">
    These values are injected as environment variables during the Jenkins
```


frontend/src/App.jsx (cont.)

```
        pipeline and baked into the Docker image.
      </p>
    </section>

    <section id="contact" className="section">
      <h3>Contact</h3>
      <p>
        Email:{" "}
        <a href="mailto:you@example.com">you@example.com</a>
        <br />
        GitHub: <a href="https://github.com/yourusername">yourusername</a>
        <br />
        LinkedIn:{" "}
        <a href="https://linkedin.com/in/yourprofile">yourprofile</a>
      </p>
    </section>
  </main>

  <footer className="footer">
    <span>? {new Date().getFullYear()} Your Name</span>
    <span>Built with React ? Deployed on Kubernetes</span>
  </footer>
</div>
);
}

export default App;
```

frontend/src/main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./index.css";

const rootEl = document.getElementById("root");

if (!rootEl) {
  console.error("Root element #root not found");
} else {
  ReactDOM.createRoot(rootEl).render(
    <React.StrictMode>
      <App />
    </React.StrictMode>
  );
}
```

frontend/src/App.css

```
.app {
  min-height: 100vh;
  color: #e5e7eb;
}

.navbar {
  position: sticky;
  top: 0;
  z-index: 20;
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 1rem 6vw;
  background: rgba(15, 23, 42, 0.9);
  backdrop-filter: blur(16px);
  border-bottom: 1px solid rgba(148, 163, 184, 0.2);
}

.logo {
  font-weight: 700;
  letter-spacing: 0.08em;
  text-transform: uppercase;
  font-size: 0.9rem;
  color: #38bdf8;
}

.navbar nav {
  display: flex;
  gap: 1.5rem;
  font-size: 0.9rem;
}

.navbar a {
  text-decoration: none;
  color: #cbd5f5;
  opacity: 0.9;
}

.navbar a:hover {
  color: #38bdf8;
}

main {
  padding: 0 6vw 4rem;
}

.hero {
  display: flex;
  align-items: center;
  min-height: 70vh;
}

.hero-content {
  max-width: 640px;
}

.tagline {
  text-transform: uppercase;
  letter-spacing: 0.16em;
  color: #9ca3af;
```

frontend/src/App.css (cont.)

```
    font-size: 0.75rem;
    margin-bottom: 0.75rem;
  }

h1 {
  font-size: clamp(2.5rem, 4vw, 3.5rem);
  margin: 0;
}

h2 {
  font-size: clamp(1.4rem, 2vw, 1.8rem);
  margin: 0.5rem 0 0.75rem;
  color: #9ca3af;
  font-weight: 500;
}

.hero-subtitle {
  max-width: 34rem;
  color: #9ca3af;
  line-height: 1.6;
}

.hero-actions {
  margin-top: 1.75rem;
  display: flex;
  gap: 0.75rem;
  flex-wrap: wrap;
}

.btn {
  padding: 0.7rem 1.4rem;
  border-radius: 999px;
  font-size: 0.9rem;
  border: 1px solid transparent;
  cursor: pointer;
  text-decoration: none;
  display: inline-flex;
  align-items: center;
  justify-content: center;
}

.btn.primary {
  background: linear-gradient(to right, #38bdf8, #6366f1);
  color: #0b1120;
}

.btn.secondary {
  border-color: rgba(148, 163, 184, 0.4);
  color: #e5e7eb;
}

.section {
  margin-top: 4rem;
}

.section h3 {
  font-size: 1.3rem;
  margin-bottom: 0.75rem;
}
```

frontend/src/App.css (cont.)

```
.section p {
  max-width: 60rem;
  color: #9ca3af;
  line-height: 1.7;
}

.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1.25rem;
  margin-top: 1.25rem;
}

.card {
  background: rgba(15, 23, 42, 0.9);
  border-radius: 1rem;
  padding: 1.2rem 1.3rem;
  border: 1px solid rgba(148, 163, 184, 0.25);
  box-shadow: 0 18px 40px rgba(15, 23, 42, 0.8);
}

.card h4 {
  margin: 0 0 0.5rem;
  font-size: 1rem;
}

.card p {
  margin: 0;
  font-size: 0.9rem;
}

.ci-panel {
  border-radius: 1.2rem;
  padding: 1.5rem 1.5rem 1.4rem;
  background: radial-gradient(circle at top left, #0f172a, #020617);
  border: 1px solid rgba(56, 189, 248, 0.5);
  box-shadow: 0 24px 60px rgba(8, 47, 73, 0.9);
}

.ci-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(210px, 1fr));
  gap: 1rem;
  margin-top: 1rem;
}

.ci-item {
  padding: 0.85rem 1rem;
  border-radius: 0.9rem;
  background: rgba(15, 23, 42, 0.9);
  border: 1px solid rgba(148, 163, 184, 0.35);
}

.label {
  display: block;
  font-size: 0.75rem;
  text-transform: uppercase;
  letter-spacing: 0.12em;
  color: #9ca3af;
  margin-bottom: 0.2rem;
}
```

frontend/src/App.css (cont.)

```
}

.value {
  font-size: 1rem;
  font-weight: 600;
  color: #e5e7eb;
}

.status {
  font-size: 0.9rem;
  color: #e5e7eb;
}

.status.error {
  color: #fecaca;
}

.note {
  margin-top: 1rem;
  font-size: 0.8rem;
  color: #9ca3af;
}

.footer {
  padding: 1.5rem 6vw 2rem;
  border-top: 1px solid rgba(148, 163, 184, 0.3);
  display: flex;
  justify-content: space-between;
  gap: 1rem;
  font-size: 0.8rem;
  color: #9ca3af;
}

@media (max-width: 768px) {
  .navbar {
    padding-inline: 1.5rem;
  }

  main {
    padding-inline: 1.5rem;
  }

  .navbar nav {
    gap: 0.9rem;
    font-size: 0.8rem;
  }

  .hero {
    min-height: 60vh;
  }

  .footer {
    flex-direction: column;
    align-items: flex-start;
    padding-inline: 1.5rem;
  }
}
```

frontend/src/index.css

```
*,
*::before,
*::after {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI",
    sans-serif;
  background: radial-gradient(circle at top left, #101827, #020617 55%);
  color: #e5e7eb;
}

a {
  color: inherit;
}
```

frontend/package.json

```
{
  "name": "portfolio-frontend",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.0",
    "react-dom": "^18.3.0",
    "vite": "^6.4.1",
    "@vitejs/plugin-react": "^4.7.0"
  }
}
```


frontend/vite.config.mts

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  build: {
    outDir: "dist",
  },
});
```

Dockerfile

```
FROM node:20-alpine AS builder
```

```
WORKDIR /app/frontend
```

```
# Build-time arguments coming from Jenkins
```

```
ARG APP_VERSION="v1.0.0"
```

```
ARG BUILD_NUMBER="0"
```

```
ARG DEPLOY_TIME="unknown"
```

```
ARG ENVIRONMENT="Kubernetes"
```

```
# Step 1: Copy only package files first (better layer caching)
```

```
COPY frontend/package.json frontend/package-lock.json* ./
```

```
# Step 2: Install ALL dependencies (vite lives in devDependencies)
```

```
RUN npm install
```

```
# Step 3: Copy source AFTER install (node_modules will NOT be overwritten)
```

```
COPY frontend/ ./
```

```
# Step 4: Create build-info.json that the React app will fetch at runtime
```

```
RUN mkdir -p public && \
  printf '{\n  "appVersion": "%s",\n  "buildNumber": "%s",\n  "deployTime": "%s",\n  "environment": "%s"\n}\n' \
  "$APP_VERSION" "$BUILD_NUMBER" "$DEPLOY_TIME" "$ENVIRONMENT" > public/build-info.json
```

```
# Step 5: Build production bundle
```

```
RUN npm run build
```

```
# --- Runtime image ---
```

```
FROM nginx:stable-alpine
```

```
COPY --from=builder /app/frontend/dist /usr/share/nginx/html
```

```
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

nginx.conf

```
server {
    listen 80;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    # Gzip compression
    gzip on;
    gzip_types text/plain text/css application/json application/javascript text/xml
    application/xml application/xml+rss text/javascript;

    # React Router ? send all routes to index.html
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Cache static assets
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
```

Jenkinsfile

```
pipeline {
    agent any

    environment {
        DOCKERHUB_CREDENTIALS_ID = 'dockerhub-creds'
        KUBECONFIG_CREDENTIALS_ID = 'kubeconfig-creds'
        DOCKERHUB_REPO = '23p61a05m5/portfolio'
        DOCKERHUB_REPO_BACKEND = '23p61a05m5/portfolio-backend'
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Install & Build Frontend') {
            steps {
                dir('frontend') {
                    bat 'npm install'
                    bat 'npm run build'
                }
            }
        }

        stage('Build Backend Docker Image') {
            steps {
                dir('backend') {
                    bat "docker build -t %DOCKERHUB_REPO_BACKEND%:%BUILD_NUMBER% ."
                }
            }
        }

        stage('Build Frontend Docker Image') {
            steps {
                script {
                    env.APP_VERSION = "v1.0.${env.BUILD_NUMBER}"
                }
                bat """
                docker build ^
                --build-arg APP_VERSION=%APP_VERSION% ^
                --build-arg BUILD_NUMBER=%BUILD_NUMBER% ^
                --build-arg ENVIRONMENT=Kubernetes ^
                -t %DOCKERHUB_REPO%:%BUILD_NUMBER% .
                """
            }
        }

        stage('Push Docker Images') {
            steps {
                withCredentials([
                    usernamePassword(
                        credentialsId: env.DOCKERHUB_CREDENTIALS_ID,
                        usernameVariable: 'DOCKERHUB_USER',
                        passwordVariable: 'DOCKERHUB_PASS'
                    )
                ]) {
                    bat """
                    docker login -u %DOCKERHUB_USER% -p %DOCKERHUB_PASS%
                """
            }
        }
    }
}
```

Jenkinsfile (cont.)

```
        docker push %DOCKERHUB_REPO%:%BUILD_NUMBER%
        docker push %DOCKERHUB_REPO_BACKEND%:%BUILD_NUMBER%
        """
    }
}

stage('Deploy to Kubernetes') {
    steps {
        withCredentials([
            file(
                credentialsId: env.KUBECONFIG_CREDENTIALS_ID,
                variable: 'KUBECONFIG_FILE'
            )
        ]) {
            bat """
            set KUBECONFIG=%KUBECONFIG_FILE%
            kubectl set image deployment/portfolio
portfolio=%DOCKERHUB_REPO%:%BUILD_NUMBER%
            kubectl set image deployment/portfolio-backend
portfolio-backend=%DOCKERHUB_REPO_BACKEND%:%BUILD_NUMBER%
            kubectl rollout status deployment/portfolio
            kubectl rollout status deployment/portfolio-backend
            """
        }
    }
}
}
```

k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: portfolio
  labels:
    app: portfolio
spec:
  replicas: 2
  selector:
    matchLabels:
      app: portfolio
  template:
    metadata:
      labels:
        app: portfolio
    spec:
      containers:
        - name: portfolio
          image: 23p61a05m5/portfolio:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 80
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: portfolio-backend
  labels:
    app: portfolio-backend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: portfolio-backend
  template:
    metadata:
      labels:
        app: portfolio-backend
    spec:
      containers:
        - name: portfolio-backend
          image: 23p61a05m5/portfolio-backend:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 3000
```

k8s/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: portfolio
  labels:
    app: portfolio
spec:
  type: NodePort
  selector:
    app: portfolio
  ports:
    - name: http
      port: 80
      targetPort: 80
      nodePort: 30080
---
apiVersion: v1
kind: Service
metadata:
  name: portfolio-backend
  labels:
    app: portfolio-backend
spec:
  type: NodePort
  selector:
    app: portfolio-backend
  ports:
    - name: http
      port: 3000
      targetPort: 3000
      nodePort: 30081
```